

DOCUMENT TECHNIQUE — ALGORITHME DE TRANSBORDEMENT INTELLIGENT MÉTRO-TAXI

Pour dépôt de brevet auprès de l'INPI

Titre de l'invention : Système et procédé de transbordement intelligent pour réseau de covoiturage urbain par abonnement

Inventeur : Judée Hamadjouldé Souleymane Nazim

Date du document : Avril 2026

Marque déposée : Métro-Taxi

- Dépôt initial auprès de l'INPI : **05 février 2020**

- Modification du logo : **20 avril 2026**

1. DOMAINE DE L'INVENTION

L'invention concerne un algorithme de transport urbain qui permet à un usager d'atteindre sa destination en utilisant successivement plusieurs véhicules de covoiturage, via des points de correspondance (transbordements) calculés automatiquement en temps réel.

2. PROBLÈME TECHNIQUE RÉSOLU

Les solutions de transport existantes (Uber, Bolt, Lyft, etc.) fonctionnent sur un modèle **point-à-point unique** : un seul véhicule est assigné pour l'intégralité du trajet. Si aucun véhicule ne va dans la direction exacte de l'usager, celui-ci doit attendre ou prendre un détour.

Métro-Taxi résout ce problème en introduisant un système de **transbordement intelligent** : l'usager peut changer de véhicule en route (jusqu'à 2 correspondances), à l'image d'un réseau de métro, mais avec des véhicules privés.

3. DESCRIPTION TECHNIQUE DE L'ALGORITHME

3.1. Constantes de l'algorithme

Paramètre	Valeur	Description
SEGMENT_MIN_KM	1,5 km	Distance minimale d'un segment de trajet
SEGMENT_MAX_KM	3,0 km	Distance maximale avant suggestion de transbordement
MAX_PICKUP_DISTANCE_KM	2,0 km	Distance maximale de prise en charge
MAX_TRANSFERS	2	Nombre maximum de transbordements autorisés
DIRECTION_THRESHOLD	60/100	Seuil minimum de compatibilité directionnelle

3.2. Fonctions mathématiques fondamentales

3.2.1. Calcul de distance (Formule de Haversine)

La distance entre deux points GPS (latitude, longitude) est calculée sur la surface terrestre :

```
R = 6371 km (rayon terrestre)
a = sin²(Δlat/2) + cos(lat1) × cos(lat2) × sin²(Δlng/2)
c = 2 × atan2(√a, √(1-a))
distance = R × c
```

3.2.2. Score de compatibilité directionnelle (Similarité cosinus)

L'algorithme évalue si un chauffeur et un usager vont dans la **même direction** en comparant les vecteurs de déplacement :

```
Vecteur_chauffeur = (destination_chauffeur - position_chauffeur)
Vecteur_usager = (destination_usager - position_chauffeur)

Produit_scalaire = Vx_chauffeur × Vx_usager + Vy_chauffeur × Vy_usager
Magnitude_chauffeur = √(Vx² + Vy²)
Magnitude_usager = √(Vx² + Vy²)

Similarité_cosinus = Produit_scalaire / (Magnitude_chauffeur × Magnitude_usager)
Score_direction = (Similarité_cosinus + 1) × 50 → résultat entre 0 et 100
```

Interprétation :

- Score = 100 : Le chauffeur et l'utilisateur vont exactement dans la même direction
- Score = 50 : Directions perpendiculaires

- Score = 0 : Directions opposées
- Seuil d'acceptation : 60/100

3.2.3. Score global de correspondance (Matching)

Chaque paire chauffeur-usager reçoit un score sur 100 points composé de :

Composante	Pondération	Calcul
Distance de prise en charge	40 points max	$40 - (\text{distance_km} \times 20)$
Compatibilité directionnelle	40 points max	$\text{score_direction} \times 0,4$
Places disponibles	20 points max	$\min(20, \text{places} \times 5)$

Formule : $\text{Score_total} = \text{Score_distance} + \text{Score_direction} + \text{Score_places}$

3.3. Calcul du point de transbordement optimal

Quand la distance totale du trajet dépasse la capacité d'un seul segment (SEGMENT_MAX_KM), l'algorithme calcule un **point de correspondance** par interpolation linéaire :

```
fraction = distance_cible / distance_totale
latitude_transfert = lat_départ + (lat_arrivée - lat_départ) × fraction
longitude_transfert = lng_départ + (lng_arrivée - lng_départ) × fraction
```

Ce point est situé sur la ligne directe entre le départ et la destination, à une distance optimale du départ.

3.4. Algorithme de routage multi-transbordement

L'algorithme procède en 4 étapes :

Étape 1 — Tentative de route directe

- Recherche d'un chauffeur pouvant couvrir l'intégralité du trajet
- Condition : score de direction $\geq 70/100$
- Si trouvé → route directe, efficacité = 100%

Étape 2 — Calcul du nombre de segments

- Si distance ≤ 6 km : 2 segments (1 transbordement)
- Si distance > 6 km : jusqu'à 3 segments (2 transbordements)
- Formule : $\text{nb_segments} = \min(3, \max(2, \lceil \text{distance} / \text{SEGMENT_MAX_KM} \rceil + 1))$

Étape 3 — Calcul des points de transbordement

- La distance totale est divisée en segments égaux
- Chaque point intermédiaire est calculé par interpolation linéaire (voir 3.3)

Étape 4 — Attribution des chauffeurs par segment

Pour chaque segment, l'algorithme :

1. Recherche les chauffeurs actifs à proximité du point de départ du segment
2. Filtre par distance de prise en charge (≤ 2 km)
3. Évalue la compatibilité directionnelle avec le point d'arrivée du segment
4. Exclut les chauffeurs déjà assignés aux segments précédents
5. Trie par distance de prise en charge croissante
6. Assigne le meilleur chauffeur + garde des alternatives

3.5. Calcul de l'efficacité du trajet

$$\text{Efficacité} = (\text{distance_directe} / \text{somme_distances_segments}) \times 100$$

- 100% = trajet parfaitement optimal (route directe)
- < 100% = détours liés aux transbordements

3.6. Estimation du temps total

$$\text{Temps_total} = \sum (\text{temps_attente_chauffeur_i} + \text{temps_trajet_segment_i}) + 3\text{min} \times \text{nb_transbordement}$$

Le temps de transbordement (changement de véhicule) est estimé à **3 minutes**.

4. RECHERCHE DE PASSAGERS COMPATIBLES (Côté chauffeur)

L'algorithme fonctionne aussi en sens inverse : un chauffeur peut voir les usagers compatibles avec sa route. Le système :

1. Récupère tous les usagers ayant un abonnement actif et une position GPS
2. Filtre par distance de prise en charge (≤ 2 km)
3. Vérifie les demandes de trajet en attente
4. Évalue la compatibilité directionnelle avec la destination du chauffeur
5. Trie par score de direction décroissant, puis distance croissante

5. COMBINAISONS DE TRANSBORDEMENT

Pour un transbordement simple, l'algorithme génère toutes les combinaisons possibles de deux chauffeurs :

1. Calcule le point de transfert optimal
2. Recherche les 5 meilleurs chauffeurs pour le 1er segment (départ → transfert)
3. Recherche les 5 meilleurs chauffeurs pour le 2e segment (transfert → destination)
4. Génère jusqu'à 25 combinaisons (5×5 , en excluant les doublons)
5. Trie par temps total estimé

6. CE QUI DISTINGUE L'INVENTION

Caractéristique	Solutions existantes (Uber, Bolt...)	Métro-Taxi
Modèle de trajet	Un seul véhicule, point-à-point	Multi-véhicules avec correspondances
Transbordement	Inexistant	Jusqu'à 2 correspondances automatiques
Calcul de compatibilité	Distance seule	Distance + direction + places (score composite)
Points de correspondance	N/A	Calculés en temps réel par interpolation
Modèle économique	Course facturée au trajet	Abonnement illimité
Optimisation réseau	Individuelle	Collective (maximise le remplissage)

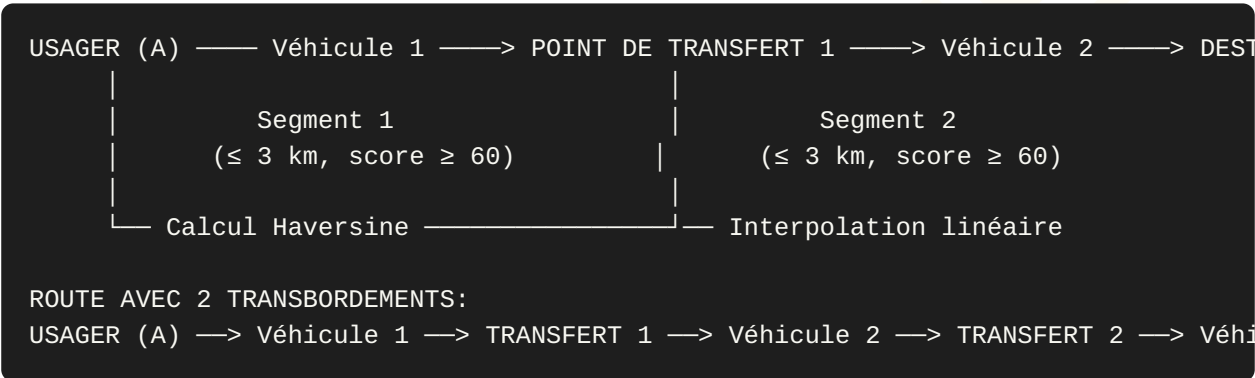
7. REVENDICATIONS

1. Procédé de transport urbain par covoiturage comprenant le calcul automatique de points de transbordement entre véhicules, permettant à un usager de changer de véhicule en route pour atteindre sa destination.
2. Procédé selon la revendication 1, caractérisé en ce que les points de transbordement sont calculés par interpolation linéaire entre le point de départ et la destination, à une distance prédéterminée (SEGMENT_MAX_KM).
3. Procédé selon les revendications 1 et 2, caractérisé en ce que la compatibilité entre un chauffeur et un usager est évaluée par un score composite comprenant la distance de

prise en charge, la similarité cosinus des vecteurs directionnels, et la disponibilité de places.

- 4. Procédé selon les revendications précédentes, caractérisé en ce que le nombre de transbordements est limité à un maximum de 2 et déterminé automatiquement en fonction de la distance totale du trajet.
- 5. Système de transport urbain mettant en oeuvre le procédé selon l'une des revendications précédentes, fonctionnant sur un modèle d'abonnement illimité.

8. SCHÉMA DE PRINCIPE



Document généré à partir du code source de l'application Métro-Taxi.
Algorithme développé et implémenté dans le fichier `/app/backend/server.py`, lignes 210-660.